Chapter 10: CodeQL Library Extension

Download this chapter as a PDF for offline reading:
Chapter-10-CodeQL-Library-Extension.pdf

The standard libraries model the common frameworks and APIs. Real codebases use internal helpers, third-party libraries, and domain-specific patterns that the standard libraries do not know about. This chapter shows how to extend the libraries with custom sources, sinks, and flow models so your queries stay accurate.

Custom Sources and Sinks

A source is any place untrusted data enters the program. A sink is any place that data becomes dangerous. When the standard libraries miss one of yours, you add it yourself.

Custom source

The simplest approach is a class that extends `DataFlow::Node`:

```
class CustomSource extends DataFlow::Node {
  CustomSource() {
    this = any(CallExpr c | c.getCalleeName() = "getUserInput").flow()
  }
}
```

(See `examples/chapter10/custom-source.ql`.)

You can then use `CustomSource` anywhere you would use a built-in source.

Custom sink

The same pattern works for sinks:

```
class CustomSink extends DataFlow::Node {
  CustomSink() {
    this = any(CallExpr c | c.getCalleeName() = "logSensitive").getArgument(0).flow()
  }
}
```

(See `examples/chapter10/custom-sink.ql`.)

In a configuration you simply write:

```
override predicate isSource(DataFlow::Node source) {
  source instanceof CustomSource or
  source instanceof RemoteFlowSource
}

override predicate isSink(DataFlow::Node sink) {
  sink instanceof CustomSink
}
```

This keeps the query readable while letting you grow the set of sources and sinks independently of any single query.

Flow Summaries and Model Packs

Many libraries are pure: they take an argument and return a transformed value, or they copy a field from one object to another. Modeling every call site by hand is tedious. A *flow summary* (also called a model) describes the relationship once.

A simplified illustration (see `examples/chapter10/flow-summary-example.ql`):

```
predicate transforms(DataFlow::Node arg, DataFlow::Node ret) {
  exists(CallExpr c |
    c.getCalleeName() = "transform" and
    arg = c.getArgument(0).flow() and
    ret = c.flow()
  )
}
```

In production you place these summaries in a *model pack*—a separate QL pack that the standard library (or your own pack) imports. The official CodeQL repositories contain many such packs; you can create your own following the same layout.

Key ideas:

- Summaries are declarative. They say "argument 0 flows to the return value" without re-implementing the library.
- They compose. Once a summary exists, every query that uses the data-flow or taint library automatically benefits.
- They belong in a library pack, not in individual queries, so they can be shared and versioned.

When the standard library is missing a model for a popular open-source package, contributing the summary upstream is the best long-term fix. For internal code, a private model pack is the right home.

Testing and Exercises

Testing models

A model is only useful if it is correct. The usual checks:

- Write a small path query that uses the new source or sink and run it against a database that contains the pattern.
- Confirm that expected flows appear and that unexpected flows do not.
- If you added a barrier or sanitizer model, verify that flow stops at the barrier.
- Keep the test queries next to the models so they travel together.

The CodeQL extension for VS Code makes this fast: you can iterate on the model and re-run the path query in seconds.

Exercises

1. Define a custom source for the return value of a function named `readConfig`.
2. Define a custom sink for the argument of a function named `sendToNetwork`.
3. Write a small path query that uses both the custom source and custom sink.
4. (Optional) Sketch a flow summary for a library function `sanitize(s)` that blocks taint.

Solutions and additional examples live in the companion repository under `examples/chapter10/`.

Next Steps

You can now extend the standard libraries so they understand the sources, sinks, and data-flow patterns that are unique to your codebase. The next chapter covers how to package those extensions (and your queries) into packs, how to organize them with suites and metadata, and how to prepare them for production use.

All examples referenced above live in the companion repository:
github.com/rod-trent/MustLearnCodeQL
under `examples/chapter10/`.

Must Learn CodeQL — Rod Trent
Companion repository: github.com/rod-trent/MustLearnCodeQL